Лабораторная работа №2.
Написание простой программы для микроконтроллера STM32

Цель работы: научиться создавать простые программы для работы с GPIO микроконтроллера STM32.

Общие сведения:

GPIO (General Purpose Input/Output) — это один из самых основных и широко используемых интерфейсов в микроконтроллерах, включая STM32 от STMicroelectronics. GPIO позволяет микроконтроллерам взаимодействовать с внешним миром, включая различные датчики, исполнительные механизмы, светодиоды и другие устройства. В этом тексте мы рассмотрим принципы работы GPIO в STM32, его конфигурацию и использование.

Режимы работы: пины GPIO в STM32 могут работать в следующих режимах:

- Input floating — вход с отключенными подтягивающими резисторами;
- Input pull-up — вход с подтяжкой к логической единице;
- Input-pull-down — вход с подтяжкой к логическому нулю;
- Analog — аналоговый вход (например, для АЦП);
- Output open-drain — выход с открытым коллектором (записали 1 — выход в высокоимпедансном состоянии, записали 0 — выход прижат внутренним транзистором к земле);
- Output push-pull — выход «тяни-толкай» (записали 1 — на выходе лог. 1, записали 0 — на выходе лог. 0);
- Alternate function push-pull — альтернативная функция в режиме «тяни-толкай»;
- Alternate function open-drain — альтернативная функция в режиме открытого коллектора;

Alternate function. В этом режиме ножкой микроконтроллера управляет внутренняя цифровая периферия, например, модуль USART.

Регистры GPIO

За функциональность GPIO отвечают регистры. Ниже представлены основные регистры GPIO:

- **GPIOx_CRL** и **GPIOx_CRH** — регистры конфигурации, содержат настройки режима (вход/выход) и частоты GPIO. Доступны на чтение и запись.
- **GPIOx_IDR** и **GPIOx_ODR** — входной и выходной регистры: в первом хранится считанное со входов порта значение, во второй записывается новое состояние выводов. GPIOx_IDR доступен только на чтение, а GPIOx_ODR — на чтение и запись.
- **GPIOx_BSRR** и **GPIOx_BRR** — регистры атомарного изменения битов в GPIOx_ODR.

GPIO_CRL

Port configuration register low (GPIOx_CRL)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIO_CRL – это конфигурационный регистр для выводов порта с номерами от 0 до 7. Каждому выводу предоставлено 4-ре бита конфигурации: 2 бита MODEy и 2 бита CNFy.

- MODEy[1:0]: Режим ножки порта, вход или выход. В режиме выхода нужно выбрать максимальную частоту переключения данной ножки, насколько понял это является оптимизацией энергопотребления порта.
  00: Вход (значение после сброса)
  01: Выход, максимальная частота 10 MHz.
  10: Выход, максимальная частота 2 MHz.
  11: Выход, максимальная частота 50 MHz.

- CNFy[1:0]: Конфигурация режима.
  В режиме входа (MODEy[1:0]=00):
  00: Analog mode — аналоговый режим (подключен к АЦП или ЦАП-у)
  01: Floating input — вход с отключенными подтягивающими резисторами (значение после сброса)
  10: Input with pull-up / pull-down — вход с подтяжкой вверх или вниз
  11: Reserved — не используется

В режиме выхода (MODEy[1:0]>00):
00: General purpose output push-pull — выход в режиме тяни/толкай
01: General purpose output Open-drain — выход с открытым коллектором
10: Alternate function output Push-pull — выход альтернативной функции режиме тяни/толкай
11: Alternate function output Open-drain — выход альтернативной функции с открытым коллектором

GPIOx_CRH

GPIOx_CRH – это конфигурационный регистр для выводов порта с номерами от 8 до 15. Тут все то же, что и для регистра **GPIOx_CRL**.

Port configuration register high (GPIOx_CRH)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

GPIOx_IDR

Port input data register (GPIOx_IDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

IDRy: в этих битах содержится входное значение соответствующего порта ввода-вывода. В зависимости от того, с каким входным пином вы работаете, в соответствии с его номером выставляется его уровень сигнала на входе (если pin 0, то IDR0 выставится в 0 или 1 в зависимости от уровня сигнала).

GPIOx_ODR

Port output data register (GPIOx_ODR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

ODRy: выходные данные порта. Здесь все то же самое, что и для IDR, только 0 или 1 выставляются микроконтроллером, то есть программно.

GPIO_BSRR

Port bit set/reset register (GPIOx_BSRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

С помощью этого регистра можно сбросить или установить любой бит регистра ODR без операций чтение-модификация-запись.

- BRy: Сбросить бит y регистра ODR порта ввода-вывода (y= 0 .. 15)
  0: не оказывает влияние на соответствующий бит ODRx

1: Сбрасывает в ноль соответствующий бит ODRx

- BSy: Установить бит у регистра ODR порта ввода-вывода (y= 0 .. 15)
  0: не оказывает влияние на соответствующий бит ODRx
  1: Устанавливает в единицу соответствующий бит ODRx

GPIO_BRR

Port bit reset register (GPIOx_BRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

С помощью этого регистра можно сбросить любой бит регистра ODR без операций чтение-модификация-запись.

- BRy: Сбросить бит у регистра ODR порта ввода-вывода (y= 0 .. 15)
  0: не оказывает влияние на соответствующий бит ODRx
  1: Сбрасывает в ноль соответствующий бит ODRx

GPIO_LCKR

Port configuration lock register (GPIOx_LCKR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Этот регистр используется для блокировки конфигурационных битов порта после записи корректной последовательности в 16 бит (LCKK) регистра. Значения битов [15:0] используется для блокировки конфигурации GPIO. Во время блокирующей последовательности в LCKK значения LCKR [15: 0] не должны меняться. Когда блокирующая последовательность была записана, конфигурация выбранных портов ввода/вывода может быть изменена только после сброса микроконтроллера. Каждый LCKy бит блокирует возможность изменения четырех битов конфигурации порта (CRL, CRH).

- LCKK: Ключ блокировки.
  0: Блокировка конфигурации порта не активна.
  1: Блокировка конфигурации порта активна. GPIOx_LCKR заблокирован до следующего сброса микроконтроллера.
- LCKy: Эти биты могут быть прочитаны и записаны, но запись можно произвести только если бит LCKK равен нулю.

0: Конфигурация пина номер у не заблокирована
1: Конфигурация пина номер у заблокирована

Библиотека HAL

Для упрощения работы с GPIO, разработчики часто используют библиотеки, такие как HAL или LL.

В рамках программирования через STM32CubeIDE мы будем использовать библиотеку **HAL**.

HAL (Hardware Abstraction Layer) — это библиотека для создания приложений на stm32, разработанная компанией ST в 2014 году. HAL позволяет абстрагироваться от работы с регистрами и другим нижнем уровнем. Грубо говоря, HAL это обёртка над низкоуровневыми операциями. Конечно же это не отменяет необходимости понимания устройства микроконтроллеров, но значительно снижает уровень вхождения.

Любая работа с периферией через HAL проводится путем вызова функций, имена которых выстраиваются по шаблону: **HAL_[периферия]_[действие]**

- периферия: GPIO, I2C, SPI и пр.;
- действие: Init, Deinit, Toggle, Write, Read, Transmit, Receive и пр.

Использование HAL в GPIO

TypeDef структура

```
typedef struct
{
  uint32_t Pin;        // номер вывода
  uint32_t Mode;       // режим
  uint32_t Pull;       // режим подтягивающего резистора
  uint32_t Speed;      // скорость переключение выходного сигнала
} GPIO_InitTypeDef;
```

- *Pin* – номер вывода. Может принимать значения:
  GPIO_PIN_0
  . . . .
  GPIO_PIN_15
  GPIO_PIN_All
  GPIO_PIN_MASK

Можно использовать *маску битов* (например: GPIO_PIN_1 | GPIO_PIN_8 | GPIO_PIN_14) или все выводы сразу GPIO_PIN_MASK.

- *Mode* – режим вывода.
  GPIO_MODE_INPUT – свободный вход, брошенный в воздухе.
  GPIO_MODE_OUTPUT_PP – активный выход (Push Pull).
  GPIO_MODE_OUTPUT_OD – выход с общим стоком.
  GPIO_MODE_AF_PP – активный выход альтернативной функции.
  GPIO_MODE_AF_OD – выход с общим стоком альтернативной функции.

GPIO_MODE_AF_INPUT – вход альтернативной функции.
GPIO_MODE_ANALOG – аналоговый вход.
GPIO_MODE_IT_RISING – внешнее прерывание по положительному фронту.
GPIO_MODE_IT_FALLING – внешнее прерывание по отрицательному фронту.
GPIO_MODE_IT_RISING_FALLING – внешнее прерывание по положительному/отрицательному фронту.
GPIO_MODE_EVT_RISING – внешнее событие по положительному фронту.
GPIO_MODE_EVT_FALLING – внешнее событие по отрицательному фронту.
GPIO_MODE_EVT_RISING_FALLING -внешнее событие по положительному/отрицательному фронту.

- Pull – режим подтягивающего резистора.
  GPIO_NOPULL – резистор отключен.
  GPIO_PULLUP – резистор подключен к шине питания.
  GPIO_PULLDOWN – резистор подключен к земле.
- Speed – максимальное быстродействие переключения выходного сигнала.
  GPIO_SPEED_FREQ_LOW – низкая.
  GPIO_SPEED_FREQ_MEDIUM – средняя.
  GPIO_SPEED_FREQ_HIGH - высокая.

HAL_GPIO_Init

*void HAL_GPIO_Init (GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_Init)*

Инициализация портов ввода-вывода.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- GPIO_Init – указатель на структуру параметров инициализации типа GPIO_InitTypeDef.

Возвращаемого значения нет.

Пример инициализации вывода.

```
GPIO_InitTypeDef GPIO_InitStruct = {0};
/* конфигурация вывода PC13 на активный выход */
GPIO_InitStruct.Pin = GPIO_PIN_13; // вывод
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // режим - выход
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_MEDIUM; // средняя скорость
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
```

HAL_GPIO_DeInit

*void HAL_GPIO_DeInit (GPIO_TypeDef * GPIOx, uint32_t GPIO_Pin)*

Функция деинициализации вывода. Устанавливает конфигурацию вывода, как при сбросе микроконтроллера.

- *GPIOx* – выбор порта (GPIOA, GPIOB, GPIOC …).
- *Pin* – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).

Возвращаемого значения нет.

Пример.
HAL_GPIO_DeInit(GPIOC, GPIO_PIN_13); // сброс конфигурации PC13

HAL_GPIO_ReadPin
*GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)*
Читает состояние вывода.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- Pin – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).

Возвращает значение вывода:

- GPIO_PIN_SET – высокий уровень на выводе;
- GPIO_PIN_RESET – низкий уровень на выводе.

Пример чтения состояния порта.

```
if( HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_10) == GPIO_PIN_SET ) {
        // на выводе PC10 высокий уровень
}
else {
        // на выводе PC10 низкий уровень
}
```

HAL_GPIO_WritePin
*void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)*
Устанавливает состояние вывода.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- Pin – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).
- PinState – состояние вывода:
  GPIO_PIN_SET – высокий уровень;
  GPIO_PIN_RESET - низкий уровень.

Функция работает с регистрами битовых операций портов. Нет опасности, что прерывание может вклиниться в момент выполнения операции. Как это может случиться при использовании регистра вывода данных в режиме чтение, модификация, запись.

Пример.

HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET); // сброс вывода PC13

HAL_GPIO_WritePin(GPIOC, GPIO_PIN_10 | GPIO_PIN_12, GPIO_PIN_RESET); // сброс выводов PC10 и PC12

HAL_GPIO_TogglePin
*void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)*
Инвертирует состояние вывода.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- Pin – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).

Возвращаемого параметра нет.

Пример.

HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13); // инверсия вывода PC13

HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_3 | GPIO_PIN_5 | GPIO_PIN_10); // инверсия выводов PB3, PB5 и PB10

HAL_GPIO_LockPin

*HAL_StatusTypeDef HAL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)*

Блокировка (защита от изменения) регистров конфигурации портов.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- Pin – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).

Возвращаемого параметра нет.

Пример.

HAL_GPIO_LockPin(GPIOC, GPIO_PIN_13); // блокировка конфигурации вывода PC13

HAL_GPIO_GetState

*HAL_StatusTypeDef HAL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)*

Блокировка (защита от изменения) регистров конфигурации портов.

- GPIOx – выбор порта (GPIOA, GPIOB, GPIOC …).
- Pin – номер вывода (GPIO_PIN_0 … GPIO_PIN_15).

Возвращаемого параметра нет.

Пример.

HAL_GPIO_LockPin(GPIOC, GPIO_PIN_13); // блокировка конфигурации вывода PC13

Задание:

1. Создать отдельный каталог «simple_math» в дереве проекта. Перенести код функций add, sub, mul, div из лабораторной работы №1 в отдельный каталог под названием “simple_math”.
2. Написать программу с применением функций из библиотеки simple_math, которая делает следующее:

   а) После выполнения функции add, мигает светодиодом один раз.

   б) После выполнения функции sub, мигает светодиодом два раза.

   в) После выполнения функции mul, мигает светодиодом три раза.

   г) После выполнения функции div, мигает светодиодом четыре раза.

   Период мигания светодиода - 300 мсек. Период между операциями – 1 сек.

   Примечание: в качестве светодиода, вы можете использовать светодиод LD2 на плате NUCLEO-F103RB. Пин, который нужно задействовать для LD2: **LD2_Pin** (пин светодиода LD2 на плате NUCLEO-F103RB)

   Подсказка: для меньшего количества кода рекомендуется завести отдельную функцию мограния светодиодом.

3. Реализовать программу, которая при нажатии и удерживании кнопки зажигает светодиод. При отпускании кнопки, светодиод должен погаснуть.

   Примечание: в качестве светодиода, вы также можете использовать светодиод LD2 на плате NUCLEO-F103RB. А в качестве кнопки вы можете использовать синюю пользовательскую кнопку (user button) B1. Пин, который нужно задействовать для B1: **B1_Pin** (пин синей кнопки на плате NUCLEO-F103RB)

Обязательное условие выполнения заданий: написанный код должен сохраниться после перегенерации проекта!

4. Представить проделанные шаги 1-3 в отчёте. Также приложить к ответу файл main.c и файлы (.c и .h) библиотеки «simple_math»